Relatório Etapa 1
Trabalho de Sistemas Distribuídos e Tolerantes a Falhas

Gabriel Welter, Giovanni Milanez, Henrique Trein e Vicente Isatto

12/10/2026

Considerações iniciais

Para realização da primeira etapa do trabalho, decidimos usar o protocolo *stop-and-wait* com identificadores sequenciais de requisição, retransmissão por timeout e confirmação explícita por ACK. Dessa forma, o sistema consegue lidar com perdas, duplicações e reordenações de datagramas.

Subserviço de descoberta

O subserviço de descoberta permite que o cliente encontre o endereço IPv4 do servidor sem recebê-lo explícitamente por parâmetro, sendo necessária apenas da porta UDP pela linha de comando. Para isso, o cliente envia uma mensagem de descoberta utilizando o método de broadcast.

No cliente, o `subservico_descoberta` cria uma mensagem do tipo `MSG_DISCOVERY` pela função `protocolo.pack_discovery()`, e a envia por meio de `<broadcast>` para a porta informada (também foi incluído o envio para `127.0.0.1` visando facilitar testes locais). Após isso, o cliente espera uma resposta do tipo `MSG_DISCOVERY_RESP`, e ao recebê-la, considera que o IP de origem do pacote é o endereço do servidor.

No servidor, o `subservico_descoberta` trata mensagens de descoberta. Quando ele recebe uma mensagem de um endereço IP ainda não registrado, cria uma entrada na tabela de clientes com `last_req`, `last_num_reqs` e `last_total_sum` (todos inicializados em zero), e em seguida, envia para ele uma mensagem de resposta `MSG_DISCOVERY_RESP` via unicast.

Subserviço de processamento

No cliente, a função de processamento empacota o par `(id_req,valor)`, envia o datagrama e configura o socket com timeout de 10 milissegundos (valor escolhido para evitar uma variabilidade referente ao RTT). Se o ACK não chegar nesse intervalo, o mesmo pacote é enviado novamente. O identificador só é incrementado depois que chega um ACK com o mesmo `id_req`.

No servidor, a função de processamento compara o identificador recebido com o esperado para o IP daquele cliente. Se são iguas, o servidor atualiza `num_reqs` e `total_sum`, grava os últimos valores enviados para aquele cliente e responde com ACK.

Se a requisição já havia sido processada, o servidor não realiza a soma. Ele reenvia o último ACK salvo para o endereço da mensagem recebida e imprime a linha com `DUP!!`. Se chega uma requisição com identificador maior que o esperado, o servidor também responde com o último ACK conhecido, deixando o cliente perceber que precisa retransmitir uma requisição anterior.

Subserviço de interface

No servidor, deixamos a impressão das mensagens em funções separadas para inicialização, novas requisições e duplicatas. Isso evita misturar formatação de saída com a lógica de atualização do estado.

No cliente, a interface imprime o servidor encontrado e os dados recebidos nos ACKs. A leitura da entrada padrão fica em uma função própria, que roda em uma thread separada e envia os valores lidos para a thread principal.

Essa separação foi importante pois a leitura do usuário e o envio pela rede têm ritmos diferentes. A thread de leitura pode continuar recebendo entrada, enquanto a thread principal consegue controlar os outros aspectos (descoberta, envio, espera por ACK, timeout e fechamento do socket).

Sincronização e consistência

A sincronização explícita aparece no cliente, entre a thread leitora e a thread principal, para isso usamos a biblioteca `Queue`, já que ela permite criar uma fila que pode ser acessada por várias threads simultaneamente. Utilizamos também a mesma biblioteca para tratar o fim da entrada.

Quando a leitura termina por `CTRL+D`, a thread leitora coloca `None` na fila. Interpretamos o "manager" citado no enunciado como a thread principal do cliente; portanto, essa flag é a sinalização interna de que o usuário está saindo do serviço. Ao consumir `None`, a thread principal sai do laço, fecha o socket e encerra o processo. Para `CTRL+C`, o próprio sistema operacional interrompe a thread principal, e o fechamento do socket também acontece no bloco `finally`.

A consistência é preservada pela ordem sequencial do laço principal e pela regra de aceitar apenas o próximo `id_req` esperado por cliente.

Estruturas e funções principais

O estado do servidor fica concentrado na classe `EstadoServidor`. O campo `clientes` é um dicionário indexado por IP. Para cada cliente, guardamos `last_req`, que indica a última requisição processada; `last_num_reqs`, que indica o valor global de requisições enviado no último ACK; e `last_total_sum`, que indica a soma global enviada no último ACK.

Os dois últimos campos são reponsáveis por permitir responder a duplicatas sem recalcular ou alterar o estado global. O ACK reenviado precisa refletir a resposta da última requisição processada daquele cliente.

No cliente, as estruturas mais importantes são o socket UDP, a fila e o contador `id_req`, o qual é controlado pela thread principal e só é incrementado após confirmação. Evitando, assim, que duas requisições do mesmo cliente fiquem pendentes ao mesmo tempo.

Serialização e comunicação

Todas as mensagens passam pelo arquivo `protocolo.py` utilizam estruturas para serializar os datagramas, deixando assim o tamanho das mensagens fixo. O modificador `!` foi usado em todos os formatos para forçar *big-endian*, o que faz com que a interpretação dos campos não dependa da arquitetura do processador da máquina que está executando cliente ou servidor.

As requisições usam o formato `!BQQ`, com um byte para o tipo da mensagem, oito bytes para `id_req` e oito bytes para o valor. Os ACKs usam `!BQQQ`, acrescentando `num_reqs` e `total_sum`. No servidor, aplicamos a máscara `0xFFFFFFFFFFFFFFFF` após incrementar os contadores. Isso simula o overflow de um inteiro sem sinal de 64 bits e evita que um valor grande demais quebre o `struct.pack()`.

Decisões de projeto

Uma decisão importante foi identificar os clientes pelo IP para possibilitar o uso de múltiplas estações, ao custo de limitar testes com vários clientes na mesma máquina, uma vez que compartilham o mesmo endereço. Com o intuito de cobrir esse caso, realizamos testes em múltiplas máquinas e também criamos uma branch paralela com containers Docker simulando uma sub-rede.

Outra decisão foi manter a sinalização de saída como algo interno ao cliente. A versão principal usa `None` na fila para avisar a thread principal. Também criamos uma branch paralela em que a saída é comunicada ao servidor pela mensagem.

Por fim, priorizamos compatibilidade com Linux, já que esse é o ambiente de avaliação. Alguns comportamentos de broadcast e rede local variaram no Windows durante os testes, então usamos esse sistema apenas como apoio de desenvolvimento, não como referência final de execução.

Problemas encontrados

O primeiro problema foi interpretar a saída do cliente. Decidimos por tratar a thread principal como manager e usar `None` como mensagem de controle entre threads.

O segundo problema foi testar vários clientes locais. Como a identificação por IP faz parte da nossa implementação principal, dois clientes na mesma máquina podem ser confundidos como o mesmo participante. A solução foi validar o cenário real em máquinas diferentes e manter a alternativa com Docker para simulações de sub-rede.

Também tivemos diferenças de comportamento no Windows, principalmente nos testes de rede. Como a entrega será avaliada em Linux, os ajustes finais e a forma de execução foram pensados para esse ambiente.

Execução e testes

Para executar o servidor:

```
python3 servidor.py 4000
```

Para executar o cliente:

```
python3 cliente.py 4000
```

Além dos testes manuais, foram criados testes automatizados para o protocolo e para partes do servidor. Também há um teste de carga para enviar várias requisições em sequência e observar se a soma final e a contagem de requisições permanecem coerentes.